ITDesk — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and where to start

The same shape as DeflectDesk, over an IT knowledge base:

- `src/rag/kb_store.py` → `build_kb_store()`: load `data/kb/*.md`, embed (`OpenAIEmbeddings`), add to Chroma. `src/rag/retriever.py` → `best_match(collection, question)` → `(text, distance)`.
- `src/confidence.py` → `is_confident(distance, cutoff=0.8)`: `distance <= cutoff`. **Build and test this first** — it's the whole answer-or-escalate decision, and it needs no model.
- `src/state.py` → `SupportState(question, context, best_distance, answer, escalated)`.
- `src/graph.py` → `build_graph(model, collection, cutoff)`: retrieve → route → answer / escalate.

Build order: confidence (test it) → RAG → state → graph → main.

Hint 2 — the graph and the pause

The route is the gate; the escalate node is the human handoff:

```
route(state): "answer" if is_confident(state["best_distance"]) else "escalate"
answer_node:  (prompt | model).invoke({context, question}) -> grounded answer
escalate_node:
    human = interrupt({"question": state["question"]})   # PAUSES the graph
    return {"answer": human, "escalated": True}
```

Wire `START -> retrieve`; `add_conditional_edges("retrieve", route, {...})`; both leaf nodes `-> END`. Compile with `checkpointer=MemorySaver()` (or `SqliteSaver`) — `interrupt` needs it. Detect the pause with `app.get_state(thread).next` (non-empty = escalated), and

resume with `app.invoke(Command(resume=human), thread)` using the **same** `thread_id`. The whole file set is in the complete solution.